NAME: NIKITA WAGHMARE
PRN:202501070121
BRANCH : ENTC
DIVISION :ET21

practical 4 :
4.1.1
Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

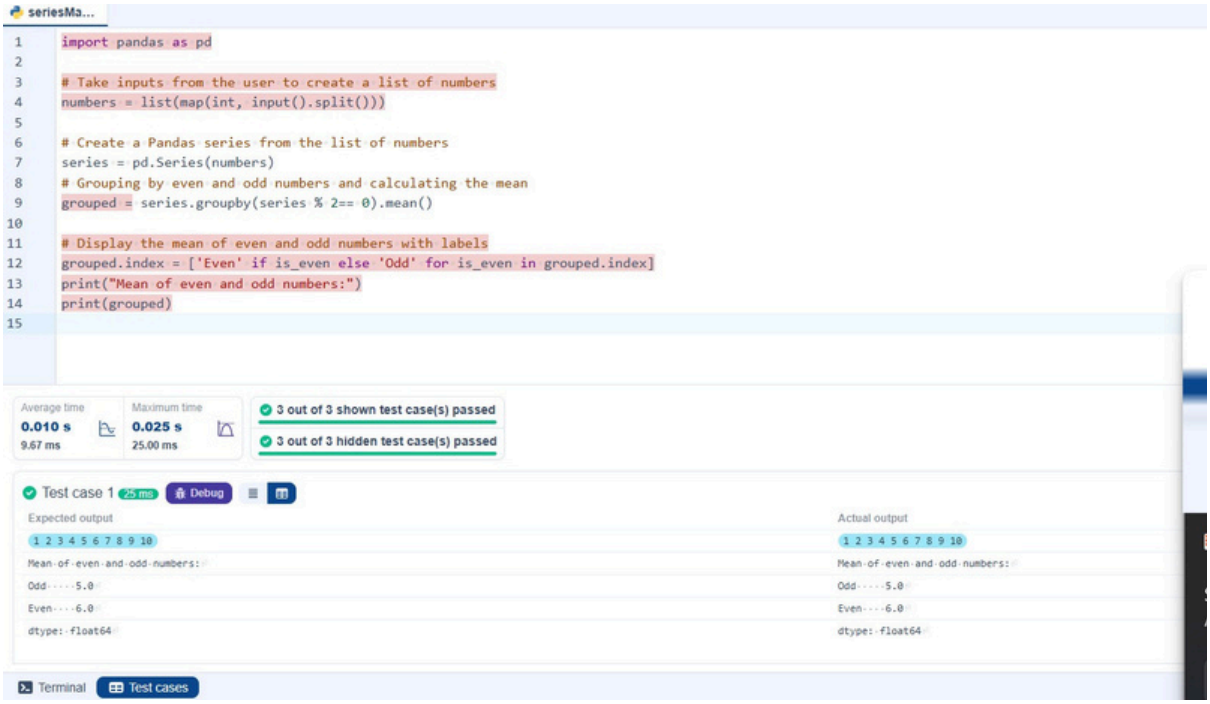
Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.
code and output :



4.1.2

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Sample Test Cases

- dataframeOps.py

```
v
#import pandas as pd
#df = pd.DataFrame(data)

## Display the original DataFrame
#print("Original DataFrame:")
#print(df)
```

```
## Adding a new row



## Display the DataFrame after adding a new row
#print("After adding a row:\n",df)

## Modifying a row



## Display the DataFrame after modifying a row
#print("After modifying a row:")
#print(df)

## Deleting a row

## Display the DataFrame after deleting a row
#print("After deleting a row:")
#print(df)

## Adding a new column



## Display the DataFrame after adding a new column
#print("After adding a new column:")
#print(df)

## Modifying a column
```

```
## Display the DataFrame after modifying a column
#print("After modifying a column:")
#print(df)

## Deleting a column

## Display the DataFrame after deleting a column
#print("After deleting a column:")
#print(df)
import pandas as pd

#Provided dictionary of lists
data = {
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35],
}

#Convert the dictionary to a DataFrame
df= pd.DataFrame(data)

#Display the original DataFrame
print("Original DataFrame:")
print(df)

#Adding a new row
new_name=input("New  name:  ")
new_age=int(input("New age: "))
```

```
new_row={'Name':new_name,'Age':new_age}
df=pd.concat([df,pd.DataFrame([new_row])],ignore_index=True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

#Modifying a row
modify_index=int (input("Index of row to modify: "))
new_age_mod=int(input ("New age: "))
df.loc[modify_index,"Age"]=new_age_mod

#Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

#Deleting a row
delete_index=int(input("Index of row to delete: "))
```

Translation Failed

Reason for late submission

Please enter at least 15 characters

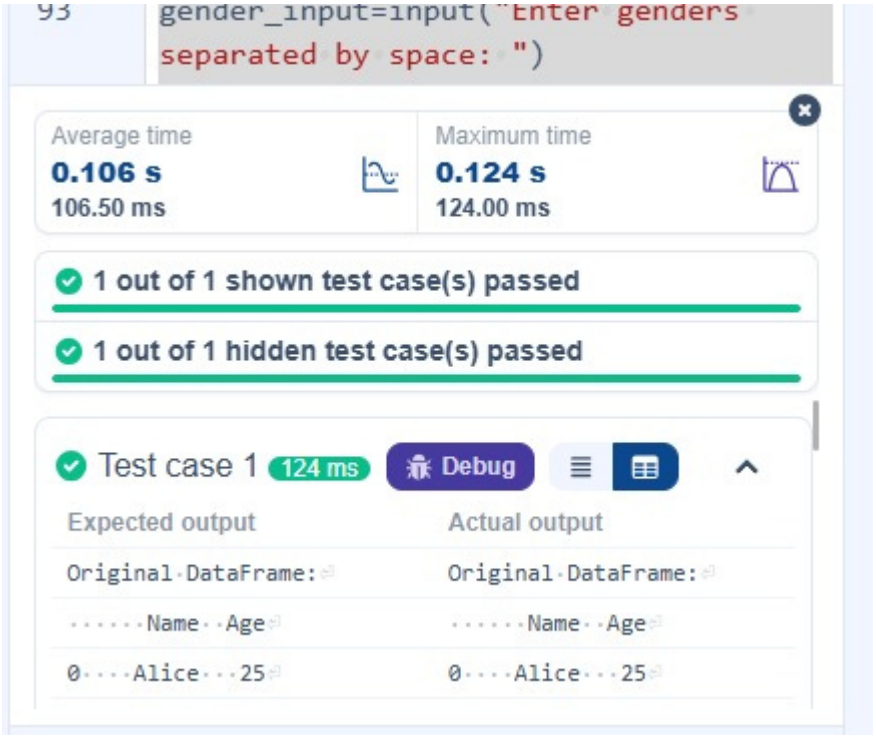
code :

```
import pandas as pd

#Provided dictionary of lists
data = {
   'Name': ['Alice', 'Bob', 'Charlie'],
   'Age': [25, 30, 35],
}
```

```
#Convert the dictionary to a DataFrame
df= pd.DataFrame(data)

#Display the original DataFrame
print("Original DataFrame:")
print(df)

#Adding a new row
new_name=input("New name: ")
new_age=int(input("New age: ")) new_row=
{'Name':new_name,'Age':new_age}
df=pd.concat([df,pd.DataFrame([new_row])],ignore_index=True)
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

#Modifying a row
modify_index=int (input("Index of row to modify: "))
new_age_mod=int(input ("New age: "))
df.loc[modify_index,"Age"]=new_age_mod

#Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

#Deleting a row
delete_index=int(input("Index of row to delete: "))
df=df.drop(delete_index).reset_index(drop=True)
```

```
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

#Adding a new column
gender_input=input("Enter genders separated by space: ")
genders=gender_input.split()
df["Gender"]= genders

#Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

#Modifying a column
df["Name"]=df["Name"].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

#Deleting a column
df=df.drop(columns=['Age'])

#Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

output :



4.1.3

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is **'B'**).

Note:

Refer to the displayed test cases for better understanding.

Code :

```
import pandas as pd


#Read the text file into a DataFrame
```

```
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

#Display the first five rows
print("First five rows:")
print(data.head())

#Calculate the average age (rounded to 2 decimal places)
average_age = round(data['Age'].mean(), 2)
print(f"Average age: {average_age}")

#Filter students with grade up to threshold 'B'
grade_order = {'A': 1, 'B': 2, 'C': 3, 'D': 4, 'F': 5}
threshold = 'B'
filtered_df = data[data['Grade'].map(grade_order) <= grade_order[threshold]]

print(f"Students with a grade up to {threshold}")
```

print(filtered_df)
output :

4.2.1
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product **A**,5,20,New York

2025-01-01,Product **B**,3,15,Los Angeles

2025-01-02,Product **A**,7,20,New York

2025-01-02,Product C,4,30,Chicago 2025-

01-03,Product **B**,2,15,Chicago 2025-01-

03,Product **A**,8,20,Los Angeles 2025-01-

04,Product C,6,30,New York 2025-01-

04,Product **B**,5,15,Los Angeles 2025-01-

05,Product **A**,3,20,Chicago 2025-01-

05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code :

```
import pandas as pd


#Prompt the user for the file name

file_name = input()


#Load the data

df= pd.read_csv(file_name)


#Convert 'Date' to datetime

df['Date'] = pd.to_datetime(df['Date'])


#Calculate total sales for each row

df['TotalSales'] = df['Quantity'] * df['Price']


#Extract month in YYYY-MM format

df['Month'] = df['Date'].dt.to_period('M')
```

#Group by month and sum total sales

monthly_sales = df.groupby('Month')['TotalSales'].sum()

#Find the month with highest total sales

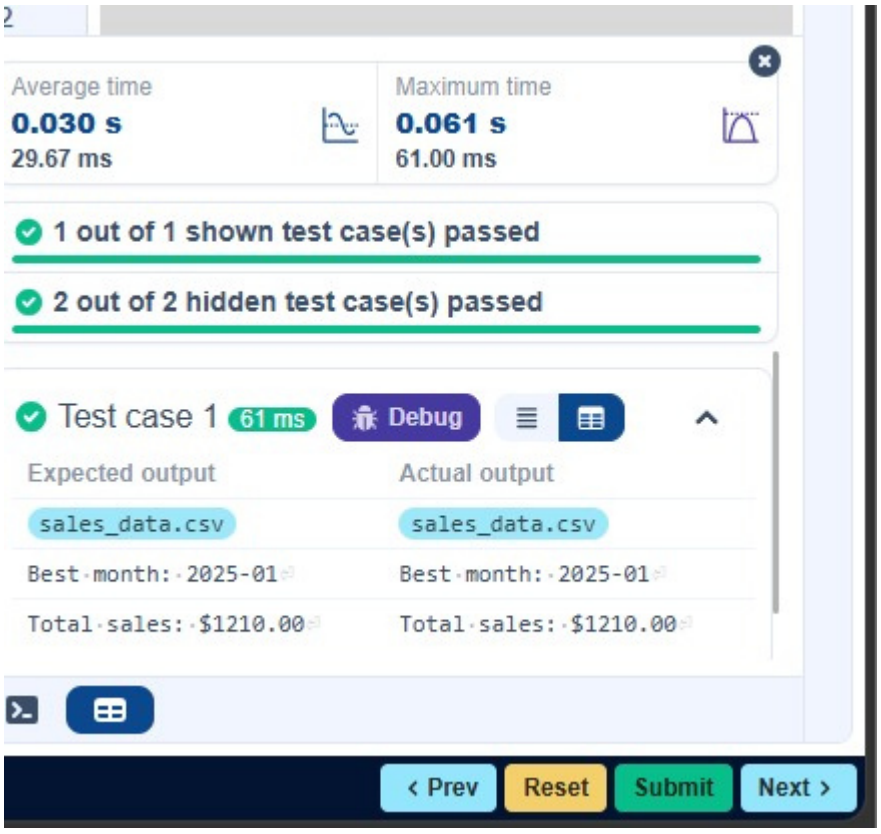
best_month = monthly_sales.idxmax()

highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")

print(f"Total sales: ${highest_sales:.2f}")

output :



4.2.2

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City 2025-01-01,Product **A**,5,20,New York 2025-01-01,Product **B**,3,15,Los Angeles 2025-01-02,Product **A**,7,20,New York 2025-01-02,Product C,4,30,Chicago 2025-01-03,Product **B**,2,15,Chicago 2025-01-03,Product **A**,8,20,Los Angeles 2025-01-04,Product C,6,30,New York 2025-01-04,Product **B**,5,15,Los Angeles 2025-01-05,Product **A**,3,20,Chicago 2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code :

```
import pandas as pd


#Prompt the user for the file name
file_name = input()
```

#Load the CSV data

df= pd.read_csv(file_name)

#Group by Product and sum the Quantity

product_sales = df.groupby('Product')['Quantity'].sum()

#Find the product with the highest quantity sold

best_product = product_sales.idxmax()

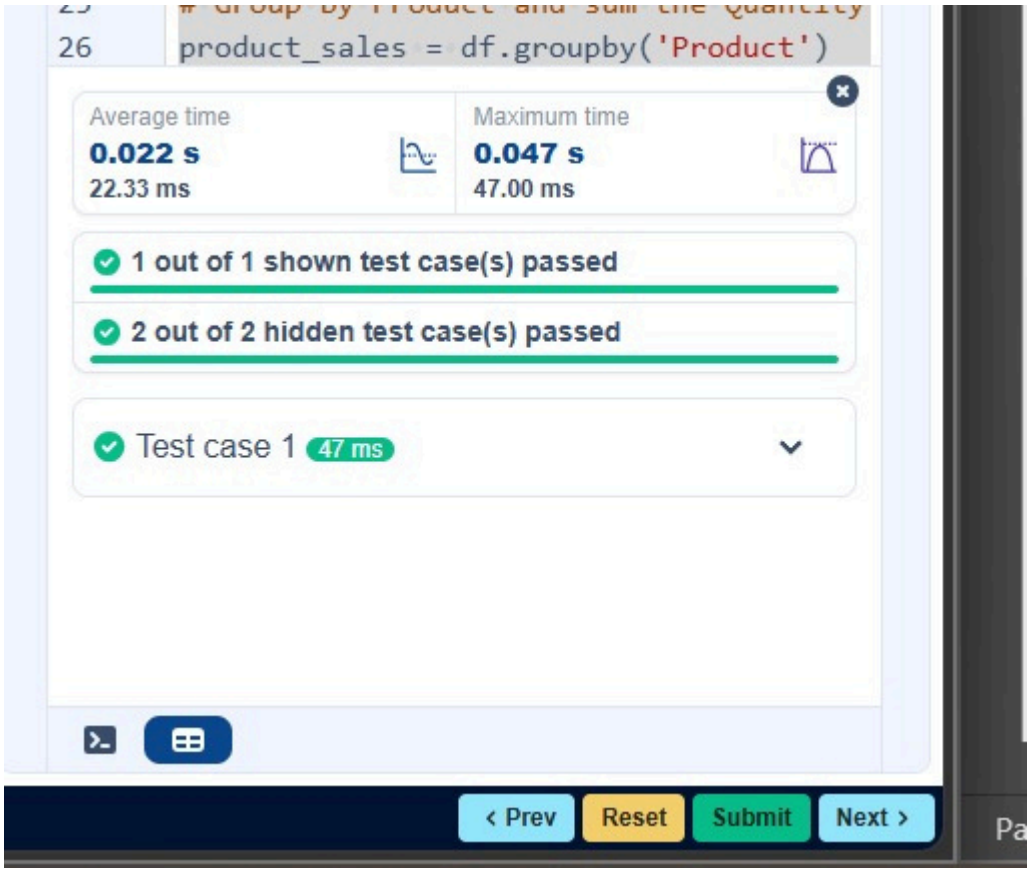
highest_quantity = product_sales.max()

#Display the result

print(f"Best selling product: {best_product}")

print(f"Total quantity sold: {highest_quantity}")

output :



4.2.3

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City 2025-01-01,Product **A**,5,20,New York 2025-01-01,Product **B**,3,15,Los Angeles 2025-01-02,Product **A**,7,20,New York 2025-01-02,Product C,4,30,Chicago 2025-01-03,Product **B**,2,15,Chicago 2025-01-03,Product **A**,8,20,Los Angeles 2025-01-04,Product C,6,30,New York 2025-01-04,Product **B**,5,15,Los Angeles 2025-01-05,Product **A**,3,20,Chicago 2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd


#Prompt the user for the file name

file_name = input()
```

#Load the CSV data

df= pd.read_csv(file_name)

#Group by City and sum the Quantity

city_sales = df.groupby('City')['Quantity'].sum()

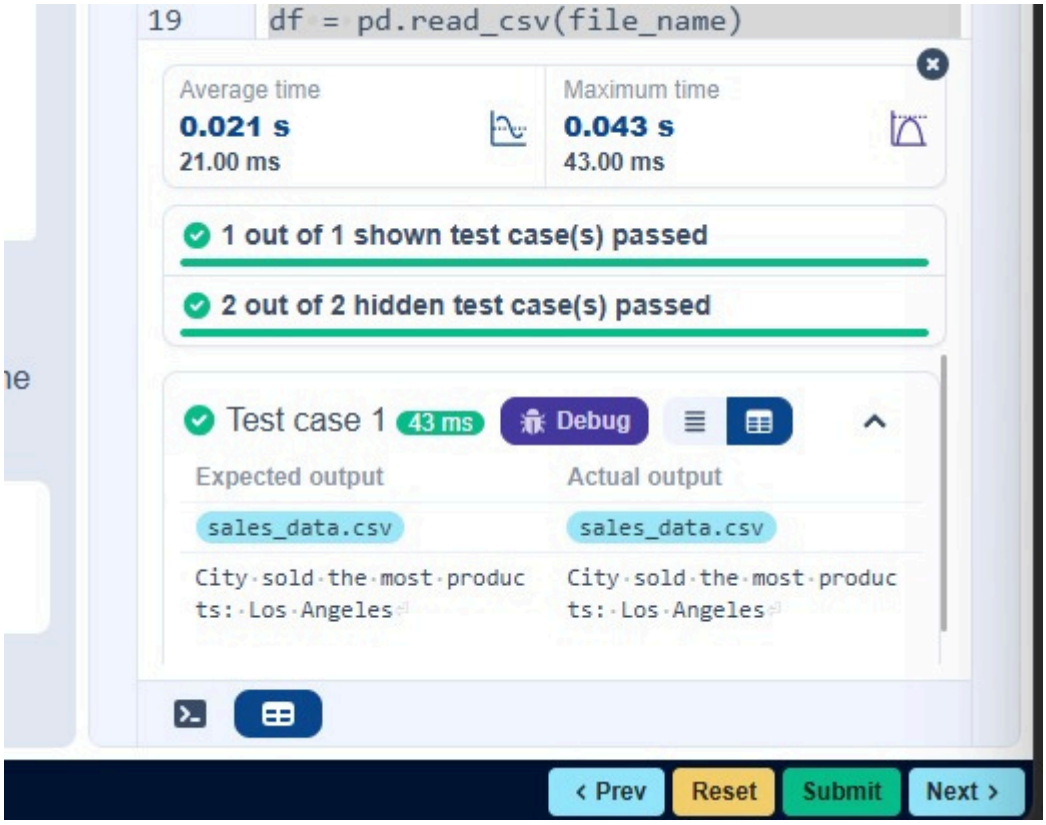
#Find the city with the highest total quantity sold

best_city = city_sales.idxmax()

#Display the result

print(f"City sold the most products: {best_city}")

output :



4.2.4

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City 2025-01-01,Product **A**,5,20,New York 2025-01-01,Product **B**,3,15,Los Angeles 2025-01-02,Product **A**,7,20,New York 2025-01-02,Product C,4,30,Chicago 2025-01-03,Product **B**,2,15,Chicago 2025-01-03,Product **A**,8,20,Los Angeles 2025-01-04,Product C,6,30,New York 2025-01-04,Product **B**,5,15,Los Angeles 2025-01-05,Product **A**,3,20,Chicago 2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B**: Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C**: Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C**: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

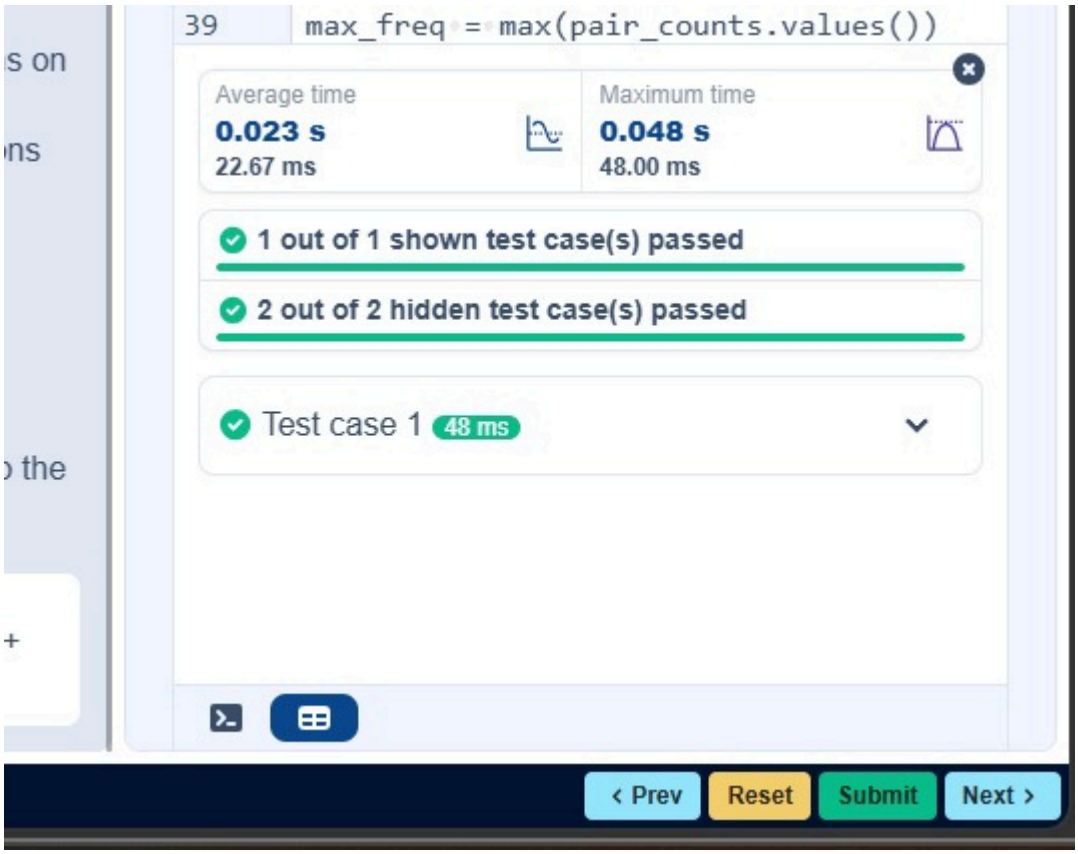
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd
from itertools import combinations
from collections import Counter


#Prompt user to input the file name
file_name = input()


#Read data from CSV
df= pd.read_csv(file_name)


#Initialize a list to store all product pairs
all_pairs = []


#Group by each date
fordate, group in df.groupby('Date'):
products = group['Product'].tolist()
#Generate all unique pairs for this date
pairs = combinations(sorted(products), 2)
all_pairs.extend(pairs)


#Count frequency of each product pair
```

```
pair_counts = Counter(all_pairs)

#Find the maximum frequency
max_freq = max(pair_counts.values())

#Get all pairs that have the maximum frequency
most_frequent_pairs = [pair for pair, count in pair_counts.items() if count == max_freq]

#Display the result in the exact expected format
forpair in most_frequent_pairs:
    print(f"{pair[0]} and {pair[1]}: {max_freq} times")
```

output:



4.2.5

You are provided with the Titanic dataset containing information about passengerson the Titanic. Your task is to write Python code to answer the following questions basedon the

dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the ‘Age’ column with the median age.
8. Fill missing values in the ‘Embarked’ column with the most frequent value (mode()).
9. Drop the ‘Cabin’ column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code :

```
import pandas as pd
import numpy as np


#Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')


#1. Display the first 5 rows of the dataset
print(data.head())


#2. Display the last 5 rows of the dataset
print(data.tail())


#3. Get the shape of the dataset
print(data.shape)


#4. Get a summary of the dataset (info)
print(data.info())


#5. Get basic statistics of the dataset
print(data.describe())


#6. Check for missing values
```

print(data.isnull().sum())

#7. Fill missing values in the 'Age' column with the median age

data['Age'] = data['Age'].fillna(data['Age'].median())

#8. Fill missing values in the 'Embarked' column with the mode

data['Embarked'] = data['Embarked'].fillna(data['Embarked'].mode()[0])

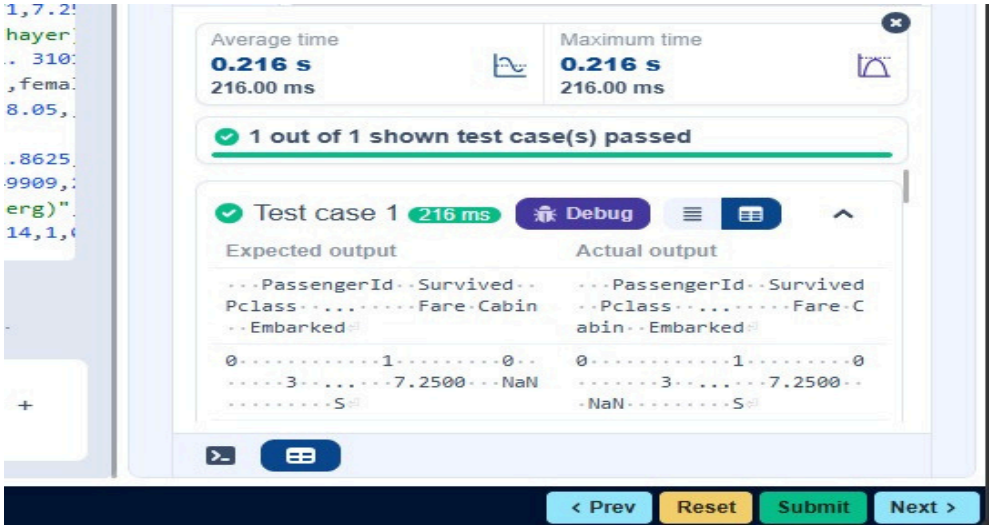
#9. Drop the 'Cabin' column due to many missing values

data = data.drop(columns=['Cabin'])

#10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

data['FamilySize'] = data['SibSp'] + data['Parch']

output :



4.2.6

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column ‘IsAlone’ which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the ‘Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the ‘Embarked’ column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
import numpy as np


#Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']


#1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = (data['FamilySize'] == 0).astype(int)


#2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})


#3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'])


#4. Get the mean age of passengers
print(data['Age'].mean())


#5. Get the median fare of passengers
print(data['Fare'].median())
```

#6. Get the number of passengers by class

print(data['Pclass'].value_counts())

#7. Get the number of passengers by gender

print(data['Sex'].value_counts())

#8. Get the number of passengers by survival status

print(data['Survived'].value_counts())

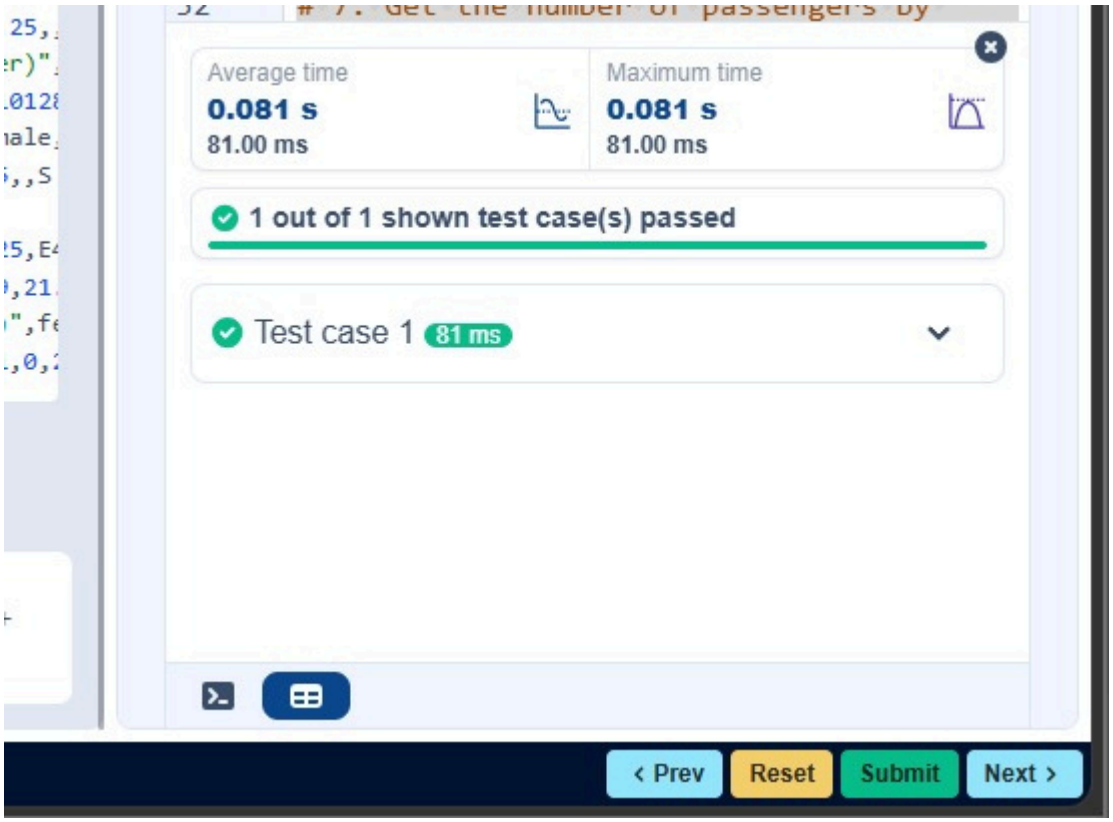
#9. Calculate the survival rate

print(data['Survived'].mean())

#10. Calculate the survival rate by gender

print(data.groupby('Sex')['Survived'].mean())

output:



4.2.7

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

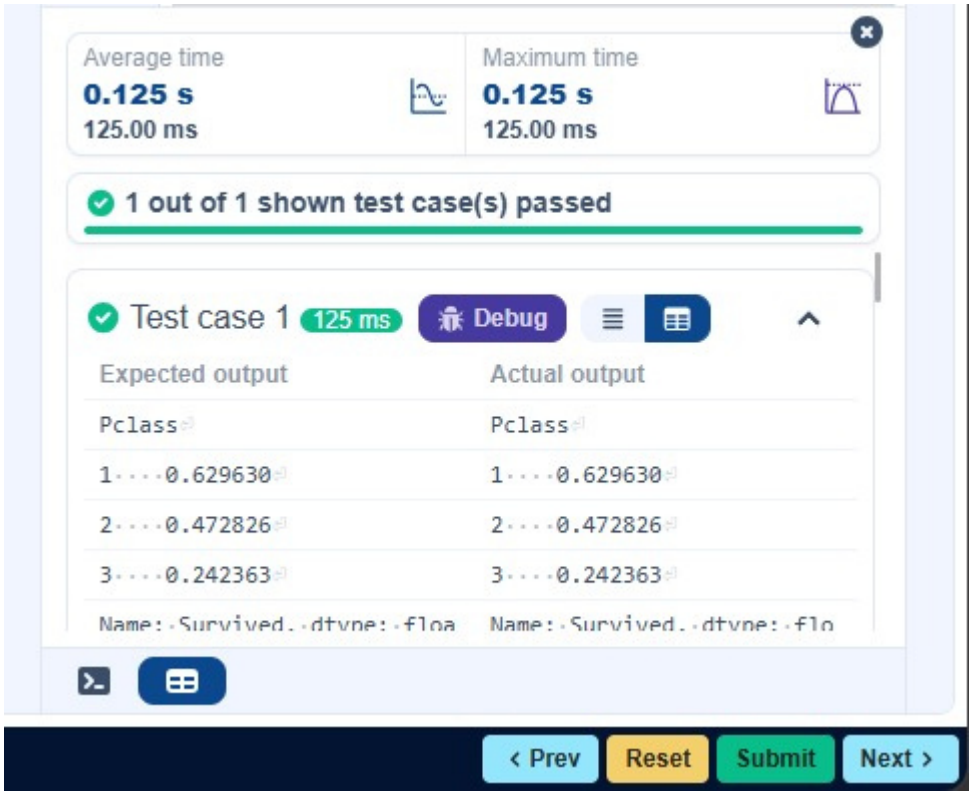
Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
import numpy as np


#Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

#1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

#2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())

#3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

#4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
print(data.groupby('Pclass')['Fare'].mean())

#6. Get the average age by class
print(data.groupby('Pclass')['Age'].mean())

#7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())

#8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())

#9. Get the number of survivors by class
print(data[data['Survived'] == 1]['Pclass'].value_counts())

#10. Get the number of non-survivors by class
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

output :



4.2.8

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

Code:

```
import pandas as pd
import numpy as np


#Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


#1. Get the number of survivors by gender
```

```
print(data[data['Survived'] == 1]['Sex'].value_counts())

#2. Get the number of non-survivors by gender
print(data[data['Survived'] == 0]['Sex'].value_counts())

#3. Get the number of survivors by embarked location
print(data[data['Survived'] == 1]['Embarked_S'].value_counts())

#4. Get the number of non-survivors by embarked location
print(data[data['Survived'] == 0]['Embarked_S'].value_counts())

#5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
print(children['Survived'].mean())

#6. Calculate the percentage of adults (Age >= 18) who survived
adults = data[data['Age'] >= 18]
print(adults['Survived'].mean())

#7. Get the median age of survivors
print(data[data['Survived'] == 1]['Age'].dropna().median())

#8. Get the median age of non-survivors
print(data[data['Survived'] == 0]['Age'].dropna().median())

#9. Get the median fare of survivors
print(data[data['Survived'] == 1]['Fare'].dropna().median())
```

10. Get the median fare of non-survivors

print(data[data['Survived'] == 0]['Fare'].dropna().median())